

Annual Report

CSR-Thematic-CPS: Action Webs Seedling

PI: Edward A. Lee
Department of EECS, UC Berkeley, eal@eecs.berkeley.edu

Award No. 0720841
Program Officer: Dr. Helen Gill

Time Period Covered by this report:
October 1, 2007 to September 30, 2008

This report and the papers it cites are available at:
<http://ptolemy.eecs.berkeley.edu/publications/actionWebs.html>

September 26, 2008

1 Activities

This project began on October 1, 2007. This report covers the first twelve months of the project, through September 30, 2008.

Currently, there are two graduate students and a postdoctoral researcher working on this project. The graduate students are Jia Zhou (a graduate of University of Minnesota, Twin Cities) and Dai Bui (a graduate of Hanoi University of Technology). Jia, now a second year graduate student, is interested in automatic code generation for real-time embedded systems. During his internship at IBM Research (summer 2008) Jia works on an implementation of the programming methodology studied within this project. Dai, who just completed his first year of graduate studies at Berkeley, is interested in parallel programming techniques for embedded systems. The postdoctoral researcher is Slobodan Matic. He recently graduated with a Ph.D. from the University of California, Berkeley and started his postdoctoral appointment on February 14, 2008. Slobodan's research interests are in models of computation and distributed and real-time embedded system design.

Christopher Brooks, the software manager and coordinator of our research group, was also partially supported by this grant. Several other graduate students of Prof. Lee are supported by different projects or fellowships and have engaged with this effort as part of their research or class work. In addition, Patricia Derler, a visiting graduate student from University of Salzburg, worked within our group on the problems relevant to this project, in particular on heterogeneous modeling of real-time systems. The collaboration with the research group of her advisor, Prof. Wolfgang Pree, will be continued in the next period of the project. In other projects related to this one we recently started collaboration with IBM Research (Java Technologies in Real-Time Systems) and HSBC (Latency Modeling in Foreign Exchange Algorithmic Trading).

2 Findings

Embedded computers monitor and control physical processes, usually with feedback loops where physical processes affect computations and vice versa. The nodes in a distributed embedded system interact both through the physical world and over a communication network. Since each computer interacts with physical processes, the passage of time becomes a central feature [?]. This is the key constraint that distinguishes these systems from distributed computing in general. The physical components of such systems introduce safety and reliability requirements qualitatively different from those in general purpose computing. The focus of this project is on fundamentals for *action webs*, and in particular on software for distributed control systems. It calls for a study of the temporal dynamics of software and networks, so that the non-idealities of software timing can be incorporated back into the control system analysis.

Software component technologies, including object-oriented design and service-oriented architectures, are built on abstractions that match software better than physical systems. The lack of temporal semantics and adequate concurrency models in computing together with best effort networking technologies make implementations nondeterministic which often leads to unpredictable and unreliable performance. For instance, abstraction mechanisms such as distributed objects often hide from programmers the safety properties of the underlying threads. In [?] we argued that it will not be sufficient to improve design processes, raise the level of abstraction, or verify designs that are built on today's abstractions. Substantial changes in the core abstractions will be necessary. We discussed promising approaches and also listed some current infrastructures that, although largely ignore timing properties, still offer innovative concurrency models.

Although similar in objective, this project is different from the PRET project (Precision Timed Architecture) of our group [?]. The goal of PRET is to introduce temporal semantics in the low-level abstractions of computing such as instruction set architectures, and to adapt architectural optimization techniques so that they deliver performance improvements without compromising temporal repeatability. In contrast to PRET, in this project we accept certain bounded uncertainty and unpredictability of the physical world and existing computing technologies, and still try to guarantee timing determinism. We also address issues such as fault-tolerance and robustness separately from scheduling, an approach different from the established technologies such as Timed-Triggered Architecture.

2.1 Programming Distributed Embedded Systems

To tackle the problems of lack of timed semantics and concurrency control in the prevailing computing and networking abstractions we proposed the PTIDES programming model (Programming Temporally Integrated Distributed Embedded Systems) [?] [?] [?]. PTIDES is based on discrete-event (DE) modeling where components send timestamped events to other components, and react to incoming events in chronological order. However, PTIDES is not a simulation technology, but rather an application specification language, which serves as a semantic basis for obtaining determinism in distributed real-time systems. Whereas for simulation all that matters is that events are processed in time-stamp order, for a real-time implementation the time instants of event processing are also important. In particular, the events that enable interaction with the environment typically involve timing constraints.

When distributed platforms are used for DE models, the objective is usually to accelerate simulation. The research in that area has focused on conservative and optimistic methods. The optimistic methods are of limited use for embedded system execution because they require roll back mechanisms which cannot be applied on actuators once they perform actions on the physical world. On the other hand, conservative methods can block event processing unnecessarily long. In such methods, it is safe to process a time-stamped event only if at no time later in the execution will an event with an earlier time stamp occur. This technique carries an unacceptably high price in our context since each platform sends messages even when there are no events to forward in order to provide lower bounds on the time stamps of future events (null messages).

The PTIDES method is along the lines of several methods that attempt to reduce or entirely eliminate the null messages. First, it binds realtime with model time (i.e., time-stamps) at sensors, actuators, and network interfaces. Second, it uses static causality analysis based on the model-time parameters of the components. This analysis determines model-time path latencies that are used during execution to check whether an event can be safely processed. It also enables independent events to be processed out of the time-stamp order. For events with mutual dependencies, the technique requires local clocks to be time-synchronized with a bounded and known error. This enables simple passage of time to be used to check if an event is safe to process, thus obviating the need for null messages. By taking advantage of dependency information, PTIDES reduces the unnecessary wait for input events and improves concurrency without resorting to an optimistic

approach that requires backtracking. Our only assumption of communication behavior is that it delivers packets reliably with a known bounded delay.

We envisioned two layers for a PTIDES implementation, the global coordination and the local scheduling layer. When an event is received (e.g. an external event from the network), the global coordination layer determines whether the event can be processed immediately or other constraints must be satisfied first according to the DE semantics. Once the event is safe to process, it is handed over to the local resource scheduler, which uses a dynamic scheduling algorithms to prioritize the processing of all pending events. In [?] we define a partial order called the relevant order that can be statically checked to enable the global coordination layer to release received events out of their time stamp order. The out-of-order execution loosens some constraints for the local resource schedulers and, therefore, allows for more irregular and flexible scheduling techniques. To refine the procedure, in [?] we presented a family of strategies that during execution determine events that are safe to process. The strategies differ in performance (i.e. event processing latency), concurrency and complexity. In addition, we developed a simulation environment for the PTIDES programming model as an experimental domain in Ptolemy II design environment.

2.2 Flexibility with Fault-Tolerance

The standard conservative technique used for distributed DE system implementation is neither fault-tolerant nor robust. A failure of a single component stops null message transmission from this component, thus blocking the progress in other components. A more flexible solution factors out time synchronization, scheduling, and fault tolerance into separable concerns.

In [?] we showed efficient creation of checkpoints that record the information necessary to recover the states of a distributed system. After the creation of a checkpoint, state changes are incrementally logged as records in constant-time operations. The applications can spontaneously restore their state by rolling back to the checkpoints. The rollback requires linear time in the number of recorded state changes, which is bounded by the number of state variables and the number of checkpoints. In [?] we apply such incremental checkpointing to introduce fault-tolerance in the PTIDES programming model. The difference between the state recovery in our approach and the one used in optimistic approaches to distributed DE execution is that the former is only performed in reaction to hardware errors, which seldom occur, while the latter is a required each time events are received

in non-chronological order. Our approach is based on automatic program transformation, which accepts the source of an existing application, and outputs the behavior-preserving application with extra functions for check-pointing.

In statically scheduled time-triggered architectures that are prevalent in fault-tolerant distributed systems the following three properties are required for timely and deterministic communication: 1) the message delay is bounded and the bounds are a priori known, 2) the receive order of messages is the same as the send order, and 3) the receive order of messages sent simultaneously is a priori known. Although the time-triggered approach is preferred for its compositionality, it is not flexible and becomes inefficient with increasing system complexity, especially if the network traffic is irregular as often the case is in the DE implementations. In [?] we achieve the similar three properties, i.e., the similar level of determinism for simple control loop applications modeled in PTIDES. In this study, the common global time is used to process the events locally, but not to control the access to the network in the static manner. We use a prototype implementation of the IEEE 1588 time synchronization protocol together with other standard networking and open-source software components. Through experiments we estimate the upper bounds of synchronization error (order of tens of nanoseconds for medium size local networks) and other event processing latencies. According to PTIDES, these estimates are important not only for control loop latencies and performance, but also to enforce deterministic behavior, since the event scheduler decisions depend on these bounds.

References

- [1] Edward A. Lee. Time is a resource, and other stories. In *International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing (ISORC)*, May 2008. Position statement for panel: "Wrong Assumptions and Neglected Areas in Embedded Research".
- [2] Edward A. Lee. Cyber physical systems: Design challenges. In *International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing (ISORC)*, May 2008. Invited Paper.
- [3] Ben Lickly, Isaac Liu, Sungjun Kim, Hiren Patel, Stephen A. Edwards, and Edward A. Lee. Predictable programming on a precision timed architecture. In *Proceedings of International Conference on Compilers, Architecture, and Synthesis from Embedded Systems*, October 2008.
- [4] Yang Zhao, Edward A. Lee, and Jie Liu. Application of programming temporally integrated distributed embedded systems. In *Proceedings of 2006 IEEE 1588 Conference*, October 2006.
- [5] Yang Zhao, Jie Liu, and Edward A. Lee. A programming model for time-synchronized distributed real-time systems. In *13th IEEE Real Time and Embedded Technology and Applications Symposium, 2007. RTAS '07*, pages 259 – 268, April 2007.
- [6] Edward A. Lee and Yang Zhao. Reinventing computing for real time. In F. Kordon and J. Sztipanovits, editors, *Proceedings of the 2006 Monterey Workshop, LNCS 4322*, pages 1–25. Springer-Verlag Berlin Heidelberg, October 2006.
- [7] Patricia Derler, Thomas Huining Feng, Edward A. Lee, Slobodan Matic, Hiren D. Patel, Yang Zhao, and Jia Zou. Ptides: A programming model for distributed real-time embedded systems. Technical Report UCB/EECS-2008-72, EECS Department, University of California, Berkeley, May 2008.
- [8] Thomas Huining Feng and Edward A. Lee. Incremental checkpointing with application to distributed discrete event simulation. In *Winter Simulation Conference (WSC 2006)*, December 2006.

- [9] Thomas Huining Feng and Edward A. Lee. Real-time distributed discrete-event execution with fault tolerance. In *14th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2008)*, April 2008.
- [10] Edward A. Lee and Slobodan Matic. Determinism in event-triggered distributed systems with time synchronization. In *Proceedings of IS-PCS07*, January 2007.